Localización y mapeo

Odometría · EKF · Landmarks · SLAM

Robótica — Capítulo 6 · Corke, *Robotics, Vision and Control*

Presiona `espacio` para avanzar →

La ruta: siete puntos de la guía

Un solo hilo conductor: estimar la pose y medir cuánto se sabe de ella.

1 · Odometría
Integrar el movimiento y ver cómo se acumula el error.

2 · EKF
Propagar también la covarianza: la incertidumbre crece.

3 · Landmarks
Corregir con un mapa conocido: la incertidumbre se acota.

4 · SLAM
Pose y mapa a la vez: sin mapa previo.

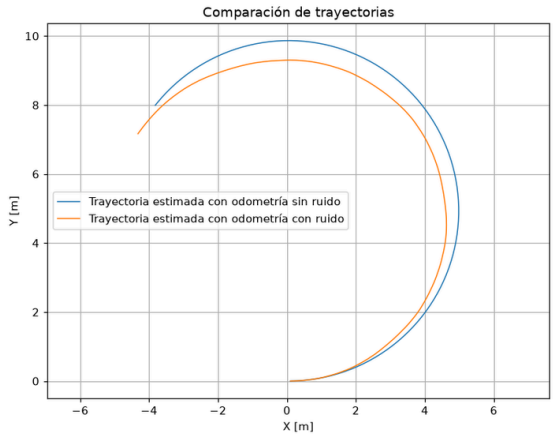
5–7 · ROS 2
Controlador, Cartographer y mapa guardado.

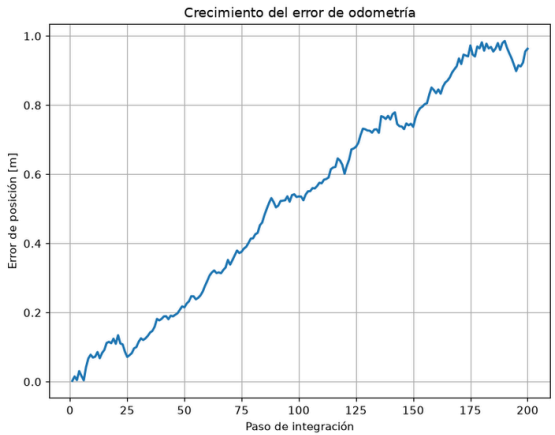
Variables
`V` odometría · P_0 inicial · `W` sensor.

Las partes 1–4 salen del mismo experimento (semilla fija), así que las figuras son comparables entre sí.

Parte 1 · Odometría: el error se acumula

← Punto de partida de todas las partes: un robot que solo integra su propia odometría.





¿Por qué la trayectoria presenta error? Cada paso suma ruido gaussiano; al integrarlo, el error se acumula y no se corrige nunca.

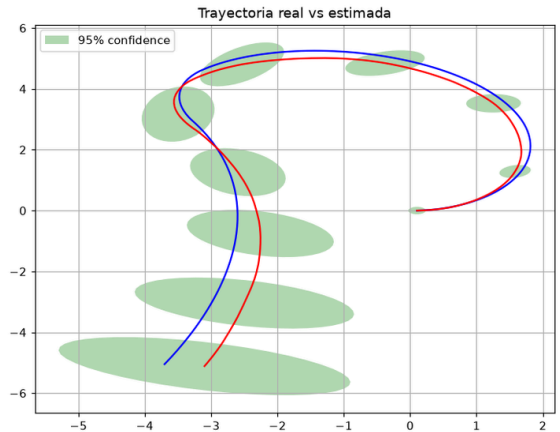
¿Qué representa `V`? La covarianza del ruido de odometría: varianzas de velocidad y de timón.

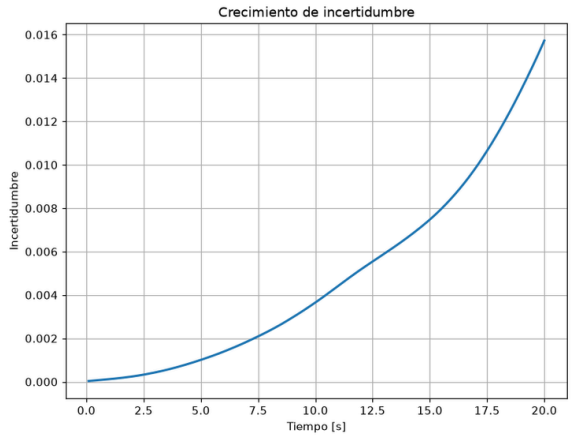
¿Qué ocurre si aumenta el ruido? La curva con ruido se separa antes y el error final crece.

Dato: 200 pasos de (1.0 m/s, 0.2 rad); error final ≈ 1 m.

Parte 2 · EKF sin sensores: la incertidumbre crece

← Si la Parte 1 acumulaba error, aquí además se propaga la covarianza del filtro.





¿Por qué el EKF no corrige el error? No hay observaciones: sin innovación, el filtro solo propaga la odometría.

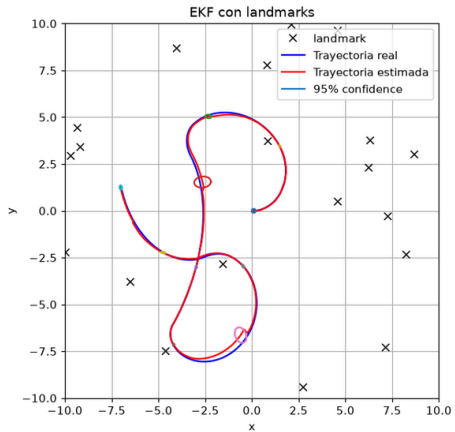
¿Qué mide la curva de la derecha? `sqrt(det P)` : crece de 3.3e-5 a 1.6e-2, unas 470 veces en 20 s.

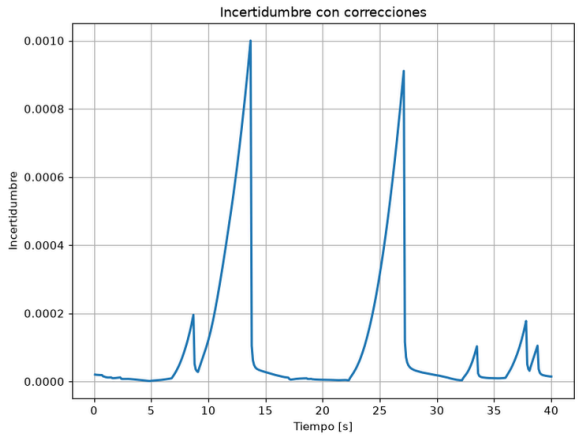
¿Qué ocurre si aumenta `V` **?** La covarianza crece más rápido; la curva se empina.

Dato: `T = 20 s` con `dt = 0.1` → 200 pasos.

Parte 3 · Landmarks: la incertidumbre se acota

← Frente a la Parte 2, ahora hay mediciones externas que corrigen la pose.





¿Qué es la innovación? La diferencia entre lo medido y lo predicho: `z - h(x)`.

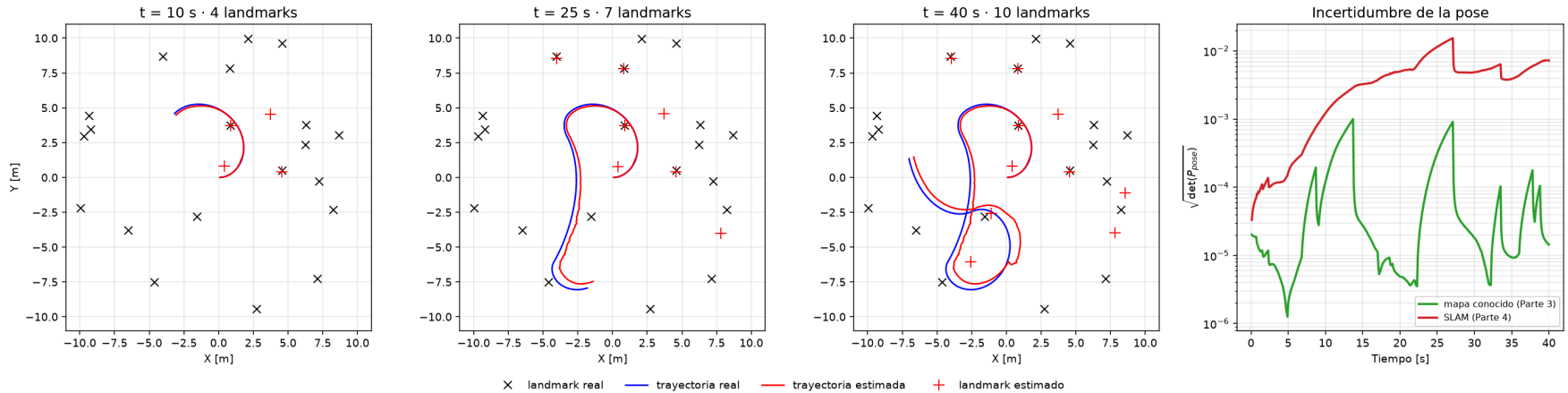
¿Por qué disminuye la incertidumbre? Cada landmark aporta información y reduce la covarianza; los dientes de sierra son esas correcciones.

¿Qué ocurre sin landmarks? Vuelve al caso de la Parte 2: la incertidumbre crece sin freno.

Dato: 20 landmarks, alcance 4 m, apertura 180°.

Parte 4 · SLAM: el mapa también se estima

← La Parte 3 conocía el mapa; aquí hay que estimar pose *y* mapa a la vez.



¿Por qué es más complejo? El estado crece `3 + 2N` y hay que asociar cada observación con su landmark.

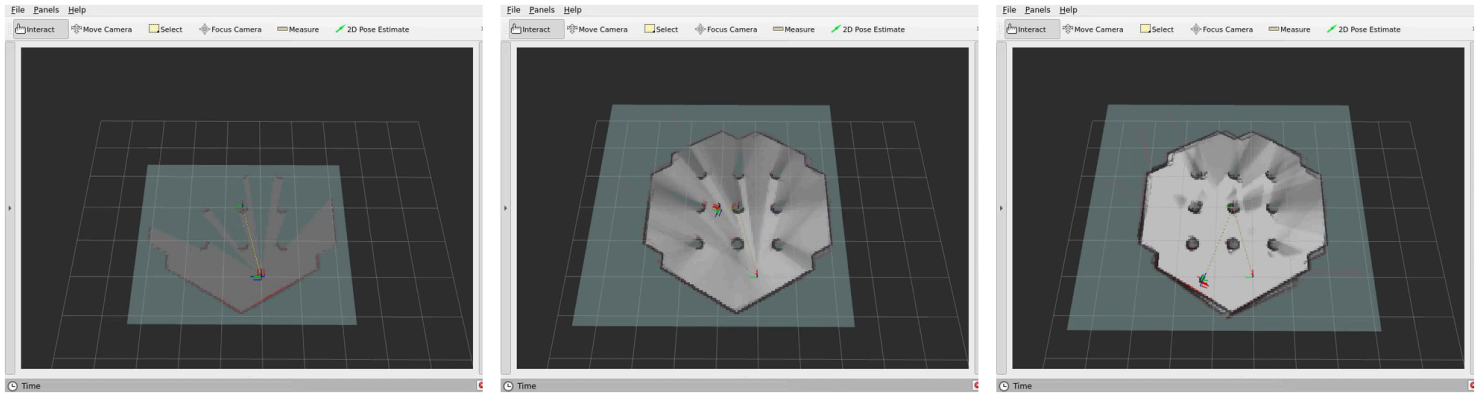
¿Por qué no desaparece la incertidumbre? Los landmarks también son incógnitas: solo se reduce al revisitar zonas conocidas.

Comparación con la Parte 3: la incertidumbre de la pose pasa de 1.4e-5 a 7.3e-3.

Ventaja: no necesita mapa previo; construye el mapa mientras se localiza.

Parte 5 · SLAM en ROS 2 con Cartographer

← Lo que en Python era un EKF, en ROS 2 es un pipeline de nodos sobre el TurtleBot3.



Pipeline: Gazebo → `/scan` , `/odom` , `/tf` → Cartographer → `/map` → RViz2.

Controlador reactivo: si el LiDAR frontal detecta < 0.5 m, gira en el sitio; si no, avanza.

Mapa guardado: `my_map.pgm` + `my_map.yaml` a 0.05 m/celda con `map_saver_cli` .

Evidencia: 110 s de grabación. El paquete del laboratorio usa ROS 2 lyrical con `TwistStamped` .

Referencias

- **Guía:** *Guía de laboratorio Capítulo 6 — Localización y mapeo*, 21 pp. (carpeta Guia/).
- **Libro:** P. Corke, *Robotics, Vision and Control*, 3.ª ed., Springer 2023, cap. 6.
- **Código del libro:** RVC3-python/notebooks/scripts/chap6.py.
- **Evidencia ROS 2:** Ref/informe_laboratorio_slam_cartographer.md y Ref/Mapeado con cartografher.webm.
- **Librerías:** roboticstoolbox-python 1.4.2 y spatialmath-python 1.1.17.
- **Revisión del código:** REPORTE_GUIA_CAP6.md.

Todas las figuras provienen de los scripts del laboratorio; no hay simulaciones en vivo.